IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2024

DISCRETE MATHEMATICS

Tuesday, 14 May 2:30 pm

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D. Goodman
Second Marker(s) : Aaron Zhao

© Imperial College London

DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   a) For each of the following statements, state whether it is true or false and provide a supporting proof.

      i) $n^2 + n = \omega(n^2)$. [3]

      ii) $(n^2+1)/(n^2-1) = O(n)$. [3]

      iii) If $f(n) = \omega(n)$ then $1/f(n) = o(1/n)$. [3]

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

      i) $T(n) = 27\,T(n/3) + n^3$. [2]

      ii) $T(n) = 26\,T(n/3) + n^3$. [2]

      iii) $T(n) = 2\,T(n/16) + 4$. [2]

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

2. This question carries 20% of the marks.

The following code is an implementation of a queue data structure. A new item is added to the queue by calling `enqueue(item)`. Calling `dequeue()` removes and returns the first remaining item that was added to the queue. The queue is implemented with list data structures that have an `append(item)` method that adds an item to the end of the list, and a `pop()` method that removes and returns the last item of the list. You can assume both operations are $O(1)$.

```
1.  in = empty list
2.  out = empty list
3.  method enqueue(item)
4.      in.append(item)
5.  method dequeue()
6.      if length(out)==0
7.          while length(in)>0
8.              out.append(in.pop())
9.      return out.pop()
```

a) Explain briefly in words how the algorithm ensures that the `dequeue()` method returns items in the order they were added by `enqueue(item)`. [2]

b) Calculate the worst case complexity of the `enqueue(item)` method. Assume that `in` has n items. [1]

c) Calculate the worst case complexity of the `dequeue()` method. Assume that `in` has n items. [2]

d) Assume that `out` is empty, `in` has n items and `dequeue()` has just been called. Compute the complexity of the worst case total amount of time spent in the `dequeue` method from now until the next time the `out` list is empty. Carefully justify your answer. [4]

e) Calculate the amortised cost of the `dequeue()` method. [1]

3. This question carries 50% of the marks.

The museum thief has done her research, and found that at the museum there are N items she would like to steal. Item $i = 1, \ldots, N$ could be sold for a price p_i, has weight w_i and would take her a total time t_i to wrap up safely so that it doesn't get damaged. She estimates she can carry a total weight W and that the alarms will go off after time T. She sets out to design an algorithm to maximise the amount of money she can make.

a) Write $P(n, w, t)$ to be the maximum amount of money she can make if she only chose from items $i = 1, \ldots, n \leq N$ with a maximum total weight of $w \leq W$ and time $t \leq T$. Assume that all weights and times are integer valued.

i) By considering the choice as to whether or not to take item n, compute a recurrence relation for $P(n, w, t)$ in terms of the values of $P(n', w', t')$ for $n' \leq n$, $w' \leq w$ and $t' \leq t$. Remember to consider the base cases. [6]

ii) Write pseudocode for an algorithm based on this recurrence relation that returns the optimal set of items to take. [6]

iii) Compute the complexity of this algorithm in terms of N, W and T. [3]

b) The museum thief is – perhaps unsurprisingly – greedy, and so she wants an algorithm that is less work. She proposes to just start with the most expensive item and keep going down the list in order of decreasing price, discarding items if she can't carry that much weight or doesn't have enough time remaining.

i) Write down an inequality for the price p_n in terms of the prices p_i for $i < n$ that guarantees that this algorithm gives the optimal choice for whether or not to take item n, whatever the values of w_i and t_i. Prove that your inequality is correct. [5]

ii) Show that if the inequality is false for some n, there is a set of weights w_i and times t_i that make her algorithm give a choice that is not optimal. [3]

iii) Assuming $p_1 = 1$ what is the smallest set of prices for which the greedy algorithm is guaranteed to be optimal? [2]